Practice · Scope and call-by-value

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What scope means

What is the **scope** of a variable?

- The type of the variable, for example `int` or `double`.
- **Right answer:** The part of the program where its name can be used.
- The largest value the variable can hold.
- The number of times the variable is changed.

Feedback: A name lives from the line that makes it to the `}` that closes its block. Made inside a function: it can be used only in that function. Made inside a loop: only in that loop. Outside its scope the name means nothing, and gcc says `undeclared`. Class 8, slides 8 and 14.

(mc) L1 · What a function gets

`main` has `int x = 5;` and calls `f(x);`. What does `f` get?

- The box `x` itself, so `f` can change `main`'s `x`.
- **Right answer:** A copy of the value 5, in its own parameter.
- Nothing: `f` must read `x` with `scanf`.
- The name `x`, but not its value.

Feedback: At a call, the value is copied into the parameter. The parameter is a new box inside the function. The function can change its copy, but the original in `main` stays the same. This is called call-by-value. Class 8, slide 17.

(mc) L1 · How a function gives back a result

A function has changed a value and `main` needs the new value. What is the way back, with what you know today?

- Change the parameter: `main` sees the change.
- Print the value inside the function.
- **Right answer:** `return` the new value, and store it in `main`: `x = f(x);`
- Give the parameter the same name as `main`'s variable.

Feedback: Changing a parameter changes only the copy, and printing is not giving back. `return` hands one value back to the caller, and `x = f(x);` keeps it. Using the same name changes nothing: they are still two boxes. Class 8, slides 25 and 29.

(mc) L2 · The parameter is set to 0

```
void g(int x) {
    x = 0;
    printf("%d ", x);
}

int main(void) {
    int x = 9;
```

```
    g(x);
    printf("%d\n", x);
    return 0;
}
```

What does it print?

- `0 0`
- **Right answer:** `0 9`
- `9 9`
- `9 0`

Feedback: Inside `g`, `x` is `g`'s own parameter, a copy of 9. It is set to 0 and printed: `0`. Back in `main`, `x` is `main`'s box, which still holds 9. Same name, two boxes. Class 8, slides 23 and 24.

(mc) L2 · A variable made inside a loop

```
int main(void) {
    for (int i = 1; i <= 3; i++) {
        int sq = i * i;
    }
    printf("%d\n", sq);
    return 0;
}
```

What happens when you compile it?

- It prints `9`.
- It prints `14`.
- It prints `0`.
- **Right answer:** It does not compile: `error: 'sq' undeclared`.

Feedback: `sq` is made inside the loop's braces, so it is gone after the loop's `}`. The `printf` below the loop cannot see it. The same rule as class 6's `int sum = 0;` inside a loop. To use a value after the loop, make the variable before the loop. Class 8, slide 13.

(mc) L2 · A function that returns

```
int add_ten(int n) {
    n = n + 10;
    return n;
}

int main(void) {
    int a = 5;
    int b = add_ten(a);
    printf("%d %d\n", a, b);
    return 0;
}
```

What does it print?

- **Right answer:** `5 15`

- `15 15`
- `5 5`
- `15 5`

Feedback: `add_ten` gets a copy of 5 in `n`, changes the copy to 15 and returns it. `b` keeps the returned 15. `a` was never touched, so it is still 5. Class 8, slide 25.

(fib) L3 · Keep the returned value

`double_it` returns its number times two. Complete the line in `main` so that `x` becomes 10. Type the whole line, with the semicolon.

```
int double_it(int n) {
    return n * 2;
}

int main(void) {
    int x = 5;
    [x = double_it(x);|x=double_it(x);|x = double_it(x) ;]
    printf("after: x = %d\n", x);
    return 0;
}
```

Feedback: `double_it(x)` gives back 10, but only `x = double_it(x);` stores it in `x`. A bare `double_it(x);` throws the 10 away and `x` stays 5. The program prints `after: x = 10`. Class 8, slide 25.

(fib) L3 · What is printed last

```
void inc(int n) {
    n++;
    printf("inside: %d\n", n);
}

int main(void) {
    int n = 3;
    inc(n);
    printf("%d\n", n);
    return 0;
}
```

The first line printed is `inside: 4`. The last line prints the number [3].

Feedback: `inc` adds 1 to its own copy, so inside it prints 4. `main`'s `n` is a different box and still holds 3. Class 8, slides 17 to 22.

(fib) L3 · A global total

```
int total = 0;

void add(int n) {
    total += n;
}
```

```
int main(void) {
    add(5);
    add(7);
    printf("%d\n", total);
    return 0;
}
```

It prints [12].

Feedback: `total` is a global variable: made outside every function, so `add` and `main` use the same box. 0 + 5 = 5, then 5 + 7 = 12. Here the function changes the global, not a copy. You will see globals in other people's code; for now, prefer parameters and `return`. Class 8, slide 15.